1.6.5 Resolution Principle

As another way of proving that an argument is correct, we use the resolution principle. This idea is used in the logic programming language Prolog.

- A variable or negation of a variable is called a **literal**.
- A disjunction of literals is called a **sum** and a conjunction of literals is called a **product**. A **clause** is a disjunction of literals, i.e. it is a sum.
- For any two clauses C_1 and C_2, if there is a literal L_1 in C_1 that is complementary to a literal L_2 in C_2 then delete L_1 and L_2 from C_1 and C_2 respectively and construct the disjunction of the remaining clauses. The constructed clause is a resolvent of C_1 and C_2.

Example 10: Suppose you have $C_1 = P \vee Q \vee R$ $C_2 = \neg P \vee \neg S \vee T$

Then P and $\neg P$ are complementary to each other. The resolvent of C_1 and C_2 is obtained by deleting P and $\neg P$ from C_1 and C_2 respectively, and finding the disjunction of the remaining literals. In this case it is $Q \vee R \vee \neg S \vee T$.

Theorem 1: Given two clauses C_1 and C_2 a resolvent C of C_1 and C_2 is a logical consequence of C_1 and C_2.

For example, consider the rules modus ponens and modus tollens.

Modus ponens rule is $P \wedge (P \rightarrow Q) \rightarrow Q$.

Putting in clause form this amounts to

$C_1 : P$

$C_2 : \neg P \vee Q$

The resolvent of C_1 and C_2 is Q which is the logical consequence of C_1 and C_2.

Similarly modus tollens rule is $(P \rightarrow Q) \wedge \neg Q \rightarrow \neg P$

Putting in clause form $C_1 : \neg P \vee Q$ $C_2 : \neg Q$